Ticket Office

Viktor Taseski
89231157
2025

Data Dictionary 1: Payment 8
Data Dictionary 2: Order 8
Data Dictionary 3: Ticket 9
Data Dictionary 4: Event 9
Data Dictionary 5: User 10
Data Dictionary 6: Organizer 10

Figure 1: Normalization 15
Figure 2: Physical Data Model 15

Diagram 1: Entity Relationship 12
Diagram 2: Use-Case 16
Diagram 3: Class 17
Diagram 4: Sequence 18

1. Problem Definition

Online ticketing platforms today often impose high service fees on every transaction and every event, driving up costs for customers and cutting into organizers' margins. And often organizers are limited by the content they can post reducing flexibility and self-promotion.

TicketOffice solves this by providing a unified, low-cost marketplace where event organizers can self-publish events directly, avoiding intermediaries and maintaining full control over ticketing. This reduces fragmentation across platforms and delivers a consistent experience for browsing, purchasing, and support. It also gives an opportunity to small businesses/organizations and individuals to have a seamless ticketing system for free.

Customers benefit from fairer pricing and a seamless, transparent purchasing process, as unnecessary service fees are eliminated.

Organizers benefit from higher profit margins, even if they choose to keep ticket prices unchanged-thanks to the significantly lower platform fees. Additionally, they gain full transparency into pricing, fulfillment costs, and real-time sales data.

Importantly, organizers retain full flexibility over ticket pricing: they can lower prices to attract more buyers, or keep them steady and capture the savings as increased profit. The platform empowers organizers with tools and freedom to tailor their ticketing strategy to their goals.

Key problems:

- Fragmented ticket sales across multiple portals, inconsistent UX (browse, purchase, support).
- Excessive service charges by intermediaries.
- Limited transparency for organizers on pricing and fulfillment costs.

My goal: a unified, low-cost marketplace where organizers publish events directly and customers enjoy fair pricing and seamless purchases. The idea is so that this flexible platform works for any gathering, for example: a private college party, live music showcase, club night, sports match, concert, movies at the cinema, festival, theatre events or anything in between. Organizers will enjoy complete creative freedom to list their events and earn profits directly, while the attendees will benefit from fair pricing and a seamless purchasing experience.

---

2. Functional Requirements

1. **Event Management (Information + Economy)**
    - **Description:** The organizers should be able to upload/update the events, change the posters of the events, give discounts to attract customers. Capture details: title, date/time, venue, pricing tiers, capacity, poster.

- **How current must the information be?** - Events must always reflect the latest updates; changes must sync instantly. Organizers can create, update, delete events. Capture details: title, date/time, venue, pricing tiers, capacity, poster.
- **What are the interfaces to external systems?** – Potential integration with venue databases or third-party event listing organizations.
- **What are the areas of the system where costs must be reduced?** – Reduce manual work by letting organizers self-manage events.
- **How much should costs be reduced or profits increased?** – Reduce admin overhead, improve organizer profits by lowering dependency on external managers.
- **What is the timetable for development?** – Critical for launch; event management is a core module.

2. **Ticket Sales & Order Processing (Performance + Efficiency + Service)**
    - **Description:** Users can buy tickets; this functionality represents the complete process from ordering to payment checkout.
    - **What is the acceptable throughput rate?** – Must handle spikes in sales (e.g., during ticket releases); ~1000+ orders per minute.
    - **What is the acceptable response time?** - Under 2 seconds for seat selection, cart updates, and checkout.
    - **Are there duplicate steps in the process that must be eliminated?** – Automate inventory updates, avoid manual confirmations. Also seat selection is repeated during order completion procedure.
    - **Are there ways to reduce waste in the way the system uses resources?** – Optimize real-time seat locking and release unused holds.
    - **Who will use thy system and where are they located?** – Customers mainly in the Balkan area, especially North Macedonia.
    - **Will there be different types of users?** – Yes, there will be visitors, registered users and organizers.
    - **What training devices and training materials are to be included?** – Customers-facing FAQs and help center content.
3. **Payment Integration (Control & Security + Performance)**
    - **Description:** A third-party system handles the payment and then a ticket is generated based on the payment status.
    - **Must access to the system or information be controlled?** – Yes, sensitive payment data must be secured.
    - **What are the privacy requirements?** – Encrypt all payment info; do not store CVV or sensitive card details.
    - **Does the criticality of the data necessitate the need for special handling?** – Yes, audit logs and transaction records need secure backup.
    - **What is the acceptable response time?** – Payment gateway responses under 2 seconds to maintain smooth checkout.
4. **User Management & Roles (Control & Security)**
    - Roles: **Customer**, **Organizer**, **Admin**. Role-based access control and audit logging. Customers have the lowest access to the system; organizers can additionally create/update their events and admins are the developers (have full access).

- **Must access to the system or information be controlled?** – Yes, strict role-based access (Customer, Organizer, Admin).
- **What are the privacy requirements?** – Personal data (emails, passwords…) must be protected.
- **Does the criticality of the data necessitate the need for special handling?** – Yes, the user role and permission data must be backed up securely.

5. **Ticket Validation & QR Code Scanning (Control & Security)**
    - **Description:** Every purchased ticket is issued with a unique QR code; on-site, organizers scan the QR to validate authenticity and mark it as used, preventing duplicate entry.
    - **What is the acceptable throughput rate?** – Must support high-traffic entry points, e.g., 300–500 scans and marks per minute per gate.
    - **What is the acceptable response time?** – QR verification must complete in under 1 second to avoid entry bottlenecks.
    - **Are there duplicate steps in the process that must be eliminated?** – Automatic "used" flagging upon first scan removes any need for manual check-off.
    - **Are there ways to reduce waste in the way the system uses resources?** – Cache recent scan results in memory to minimize repeated database hits for the same ticket.
    - **Must access to the system or information be controlled?** – Only authenticated organizer accounts may access the scan interface; all scan attempts are audit-logged.
    - **What are the privacy requirements?** – During scan, only ticket status (valid/invalid/used) is exposed—no purchaser personal data is displayed.
    - **Does the criticality of the data necessitate special handling?** – All scan events are securely logged for post-event audits; communications between scanner and backend are encrypted.
    - **What are the interfaces to external systems?** – Can integrate with mobile scanning SDKs or third-party handheld scanners via a RESTful validation API.

6. **Search & Browse (Performance + Efficiency)**
    - **Description:** This system allows the user for efficient search of events. Having keyword search with filters: category, date range, venue, price reduces searching time and gives better user experience.
    - **What is the acceptable throughput rate?** – Support concurrent searches by thousands of users.
    - **What is the acceptable response time?** – Search results under 2 seconds.
    - **Are there duplicate steps in the process that must be eliminated?** – Avoid redundant database hits; use caching.

3. Non-Functional Requirements

Performance: The system must handle at least 1,000 transactions per minute under peak load, including simultaneous ticket purchases, searches, and event updates. Seat selection, cart updates, and payment processing should complete within 2 seconds, even under heavy traffic.

Availability: The system must maintain an uptime of ≥95%, ensuring 24/7 availability for browsing, ticket purchases, event management, and reporting. Redundancy and failover mechanisms must be in place for critical services (payment processing, inventory management).

Security All communications between users, the system, and third-party services (e.g., payment gateways) must be encrypted. The system must comply with PCI DSS for payment data, follow OWASP best practices to guard against common web vulnerabilities (e.g., SQL injection, XSS), and enforce strict role-based access control (RBAC) for all user roles (Customer, Organizer, Admin). Sensitive data (user accounts, transactions) must be securely backed up and retained according to privacy laws. Ticket marking must be securely protected from unauthorized access.

Usability: The system must offer a responsive, intuitive UI that works seamlessly on both desktop and mobile devices. The platform should provide clear onboarding and help materials for both customers and organizers.

Maintainability: The system must be built on a modular, well-documented codebase that supports continuous integration and continuous deployment. Unit tests, integration tests, and automated deployment scripts must be included to ensure rapid, low-risk updates and bug fixes. New features or modules should be easy to add without major architectural changes.

Backup & Data Retention: User and transaction data must be securely stored for a minimum of 12 months. Data must always be replicated across at least two physical or cloud data carriers, with full external backups performed on a weekly basis. Events that have passed should be deleted form the database within 3 days.

Integration: The system must integrate with external services such as payment gateways (via secure APIs), venue databases, and potentially third-party event listing platforms. Tickets should support data export in standard formats (PDF).

User Types & Access: The system must support at least three distinct user roles: Customers, Organizers, and Admins. Each role should have clearly defined access rights, and the system must log all critical actions for audit purposes. Access must be possible from any internet-connected device, primarily targeting users in the Balkan region (especially North Macedonia), but scalable to support broader international expansion.

4. Feasibility Study

Technical Feasibility: Build on a microservices architecture, hosted on cloud platforms (university), uses APIs for backend communication, uses a react-based frontend for responsive and dynamic user experience, stores data in a relational DB with read replicas for scalability. **Evaluation**: These are mature, well-supported technologies with large ecosystems. The scope (event management, ticketing, payment integration, user management) fits within standard web application patterns. Cloud services ensure scalability, elasticity and high availability (meeting at least 95% uptime). Existing payment gateways (Stripe, PayPal, Adyen) offer PCI DSS-compliant integrations, reducing technical complexity. Overall, the technical foundation is realistic and implementable.

Operational Feasibility: Empower event organizers to self-publish, self-manage, and control their events. Provide automated customer-facing processes (ordering, refunds) to reduce manual intervention. **Evaluation**: Fits the market needed for unified, self-service platforms. Reduces support overhead by automating event updates and order flows. Adaptable across regions (Balkan, EU) as long as localization (language, currency 'EUR') is supported.

Economic Feasibility & Evaluation: Use open-source technology stack to reduce licensing costs. Generate revenue by charging a minimal, transparent transaction fee instead of high service charges. The platform will incorporate dedicated advertising sections where event organizers can promote their events or sponsors, providing an additional revenue stream by charging organizers per advertisement placement. Enable organizers to either pass savings to customers or retain higher margins (their choice). Furthermore, by enabling organizers to self-manage events and automating key processes (such as ticketing and inventory management), the platform significantly reduces administrative overhead and reliance on manual labor. This operational model not only cuts intermediary costs but also helps that the developer maintains a sustainable and balanced workload, focusing on improvement rather than repetitive support tasks.

Organizational/Social Feasibility: Having transparent, fair pricing builds trust with both organizers and customers. Being adaptable for multiple event types (concerts, sports, theatre, festivals) – increases social acceptability. Using role-based system (Customer, Organizer, Admin) ensures clear responsibilities and access boundaries. This system also lowers barriers for smaller/local organizers by providing an accessible publishing platform. **Evaluation:** Likely acceptable to work (organizations (venues, promoters, ticket sellers) because it reduces friction and overhead. Aligns with user expectations for digital, mobile-first experiences. Socially acceptable if the platform maintains trust through clear communication, secure handling of data, and responsive support.

---

5. Data Dictionary & Matrix of User Roles

In the following section we have descriptions of entities from the data dictionary followed by the tables of the data dictionary.
Payment: Payment transaction processed by the system.
Order: Order of tickets placed in the system (can be in cart).
Ticket: Ticket of an event (one order can have many tickets)
Event: Event listed on the system.
User: user of the system.
Organizer: user of the system who can create manage events

Payment

Attribute	Type	Description
transaction_id	int(11)	Primary key
p_order_id	int(11)	Foreign key
payment_method	varchar(100)	method of payment
amount	decimal(10,0)	payment amount
payment_date	date	date of payment
payment_status	enum('Success','Failed','Refunded','')	current status of payment

Data Dictionary 1: Payment

Order

Attributes	Type	Description
order_id	int(11)	Primary key
user_id	int(11)	Foreign key
event_id	int(11)	Foreign key
order_date	datetime	Date of order
total_amount	decimal(11,0)	Ticket price * ticket Quantity
ticketQuantity	int(11)	Number of tickets bought
payment_status	enum('Pending','Completed','Cancelled','Refunded')	Status of current payment

Data Dictionary 2: Order

Ticket

Attributes	Type	Description
ticket_id	int(11)	Primary key
order_id	int(11)	Foreign key
event_id	int(11)	Foreign key
QR_img	varchar(255)	Info for generating QR code
price	decimal(11,0)	Ticket price
seat_number	varchar(200)	Seating info for the ticket

Data Dictionary 3: Ticket

Event

Attributes	Type	Description
id_event	int(11)	Primary key
organizer_id	int(11)	Foreign key
date	date	Date of when the event is happening
time	time	Time of the event
title	varchar(150)	The title/name/topic of the event
poster	varchar(255)	A poster representing the event
e_ticket_price	decimal(10,0)	The events ticket price
availableTickets	int(11)	Number of currently available tickets (tickets left)
venue	varchar(255)	Place where the event is happening
category	varchar(100)	Category that the event belongs to
e_ticket_id	int(11)	Foreign key
description	varchar(500)	Description of the event that goes on the event card

Data Dictionary 4: Event

User

Attributes	Type	Description
id	int(11)	Primary key
email	varchar(100)	email of user
password	varchar(100)	password of user
username	varchar(100)	Name + Surname of user
userType	enum('Customer','Organizer','Admin')	The role/type of the user

Data Dictionary 5: User

Organizer

Attributes	Type	Description
registrationNumber	varchar(50)	Primary key. Every organization has their unique registration number
user_id	int(11)	Foreign key
orgName	varchar(150)	Legal name of the organization
orgType	varchar(100)	Non/profit, government etc.
taxId	varchar(50)	Tax identification number
contactEmail	varchar(100)	Organizations email for contact
contactPhone	varchar(20)	Organizations phonel for contact
address	varchar(255)	Registered address of organization
city	varchar(100)	Registered city of organization
country	varchar(100)	Registered country of organization
bankAccount	varchar(50)	Bank Account information/details

Data Dictionary 6: Organizer

Matrix of user roles

Function	Customer	Organizer	Admin
login()	Yes	Yes	Yes
register()	Yes	Yes	Yes
searchEvent()	Yes	Yes	Yes
buyTicket()	Yes	No	No
createEvent()	No	Yes	Yes
validateOrg()	No	Submit Request	Yes
updateEvent()	No	Yes	Yes
deleteEvent()	No	Yes	Yes
refundTicket()	Request Only	Request Only	Approve Only
useTicket()	No	No	Yes

Matrix of User Roles: Table 1

6. ER - Diagram

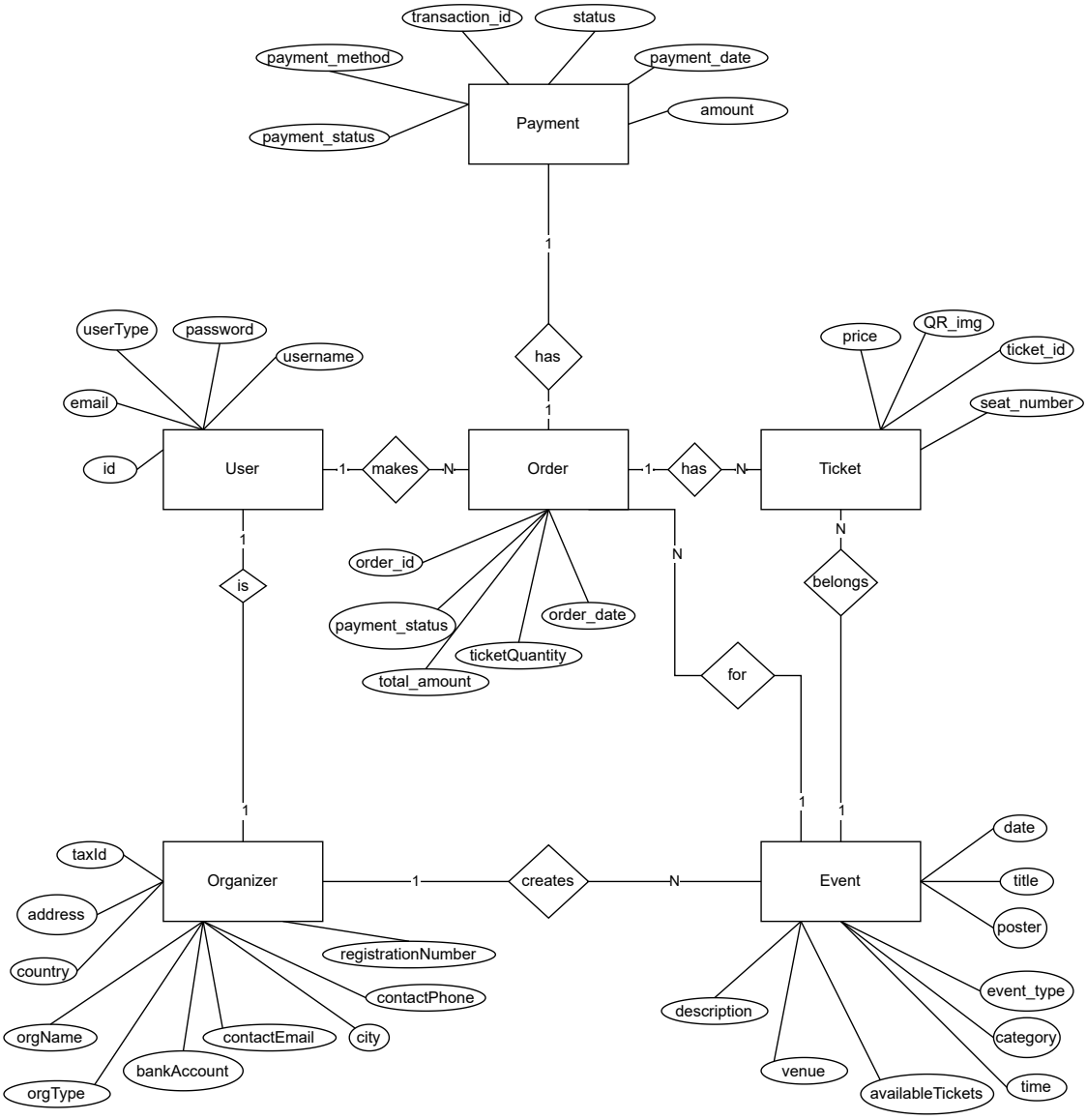

Diagram 1: Entity Relationship

6.1 Relational Model

Note: Some attributes are excluded for simplicity.

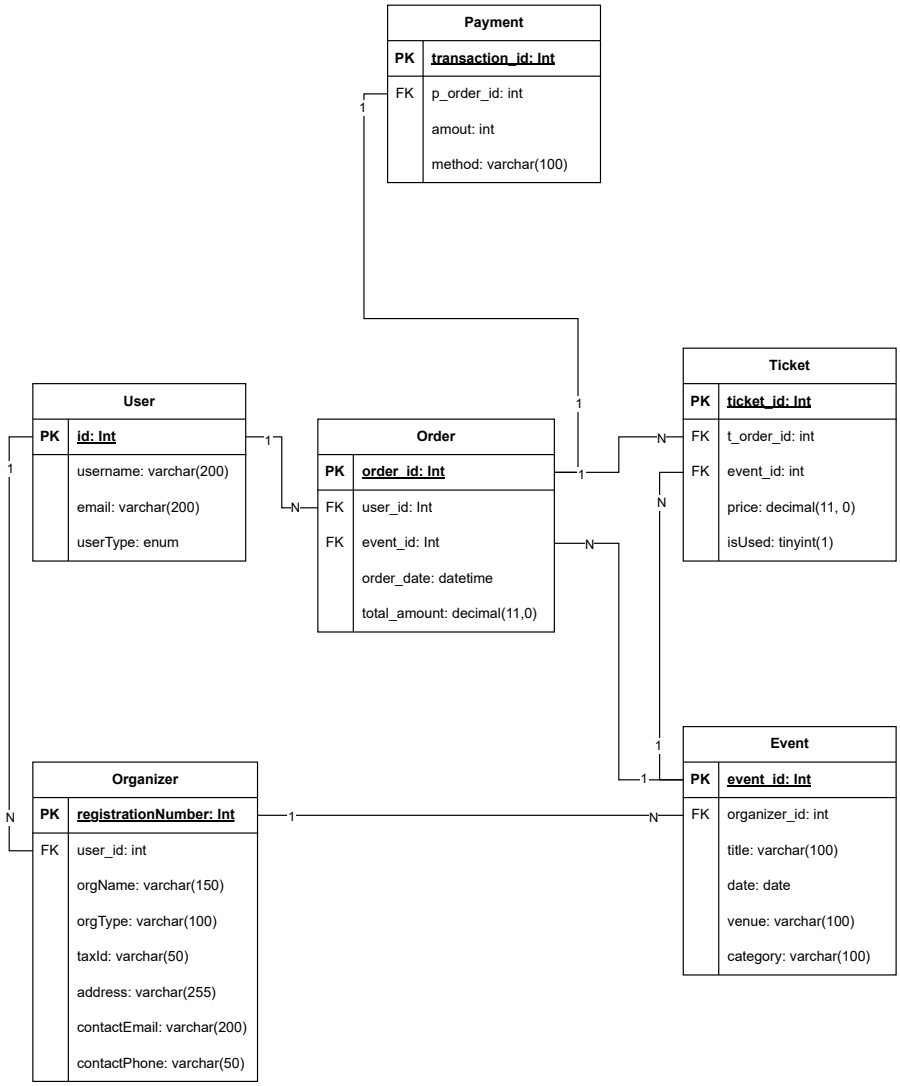

Figure 1: Relational Model

7. Relational Model

- **USER** (*user_id*, email, password, username, user_type)
- **EVENT** (*event_id*, title, event_date, event_time, venue, category, poster)
- **ORDER** (*order_id*, user_id → USER, order_datetime, total_amount, payment_status)
- **TICKET** (*ticket_id*, order_id → ORDER, event_id → EVENT, qr_code, quantity, price)
- **PAYMENT** (*payment_id*, order_id → ORDER, payment_method, amount, payment_datetime, payment_status)
- **ORGANIZER** (*registrationNumber*, user_id → USER, orgName, orgType, taxID, contactEmail, address, city, country, bankAccount)

8. Data Analysis & Normalization

USER has no composite key, and all its columns depend solely on user_id.
EVENT stores venue and category as simple fields, if later on I decide make them into their own tables I will. (if necessary).
ORDER user_id is its only foreign key, order totals and status depend on the order as a whole.
TICKET is the 'line item' linking **ORDER** and **EVENT** with its own primary key (ticket_id), plus quantity and price.
PAYMENT may record multiple settlement attempts against one order but isn't a many to many, so it lives as its own table with a simple foreign key to **ORDER**.

0NF	1NF	3NF
SALE(order_id, order_date, total_amount, payment_status, transaction_id, payment_method, amount, payment_date, payment_status, user_id, email, password, username, user_type, event_id, title, date, time, venue, category, poster, ticket_id, qr_img, quantity, price)	ORDER1(order_id, user_id, order_date, total_amount, payment_status) TICKET1(ticket_id, order_id, event_id, qr_img, quantity, price) PAYMENT1(transaction_id, order_id, payment_method, amount, payment_date, payment_status) USER1(user_id, email, password, username, user_type) EVENT1(event_id, title, date, time, venue, category, poster)	CREATE TABLE USER (user_id INT AUTO_INCREMENT PRIMARY KEY, email VARCHAR(100) NOT NULL UNIQUE, password VARCHAR(255) NOT NULL, username VARCHAR(50) NOT NULL UNIQUE, user_type ENUM('Customer', 'Organizer', 'Admin') NOT NULL DEFAULT 'Customer'); CREATE TABLE EVENT (event_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(150) NOT NULL, date DATE NOT NULL, time TIME NOT NULL, venue VARCHAR(200), category VARCHAR(100), poster VARCHAR(255)); CREATE TABLE `ORDER` (order_id INT AUTO_INCREMENT PRIMARY KEY, user_id INT NOT NULL, order_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, total_amount DECIMAL(12,2) NOT NULL, FOREIGN KEY(user_id) REFERENCES USER(user_id)); CREATE TABLE TICKET (ticket_id INT AUTO_INCREMENT PRIMARY KEY, order_id INT NOT NULL, event_id INT NOT NULL, qr_img VARCHAR(255), quantity INT NOT NULL, price DECIMAL(10,2) NOT NULL, FOREIGN KEY(order_id) REFERENCES `ORDER`(order_id), FOREIGN KEY(event_id) REFERENCES EVENT(event_id)); CREATE TABLE PAYMENT (transaction_id INT AUTO_INCREMENT PRIMARY KEY, order_id INT NOT NULL, payment_method VARCHAR(50) NOT NULL, amount DECIMAL(12,2) NOT NULL, payment_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, payment_status ENUM('Success', 'Failed', 'Refunded') NOT NULL DEFAULT 'Success', FOREIGN KEY(order_id) REFERENCES `ORDER`(order_id));

Figure 2: Normalization

9. Physical Data Model

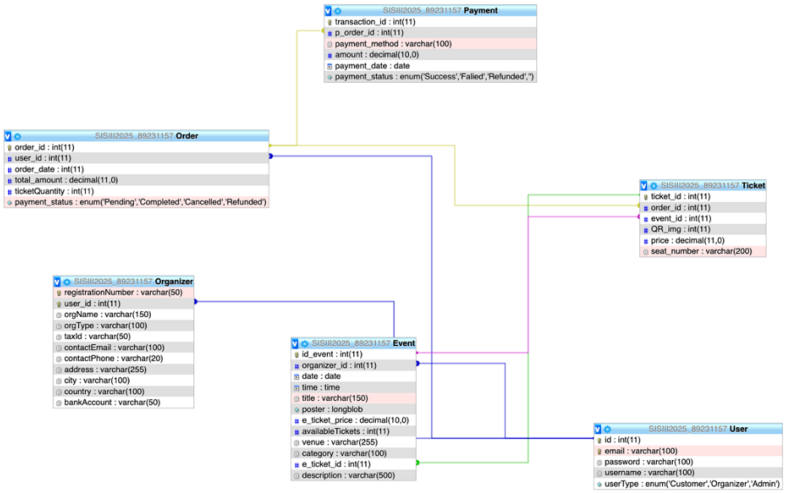

Figure 3: Physical Data Model

10. UML Diagrams

Use-case Diagram

This use-case diagram represents how the all users (customer and organizers) interact with the system.

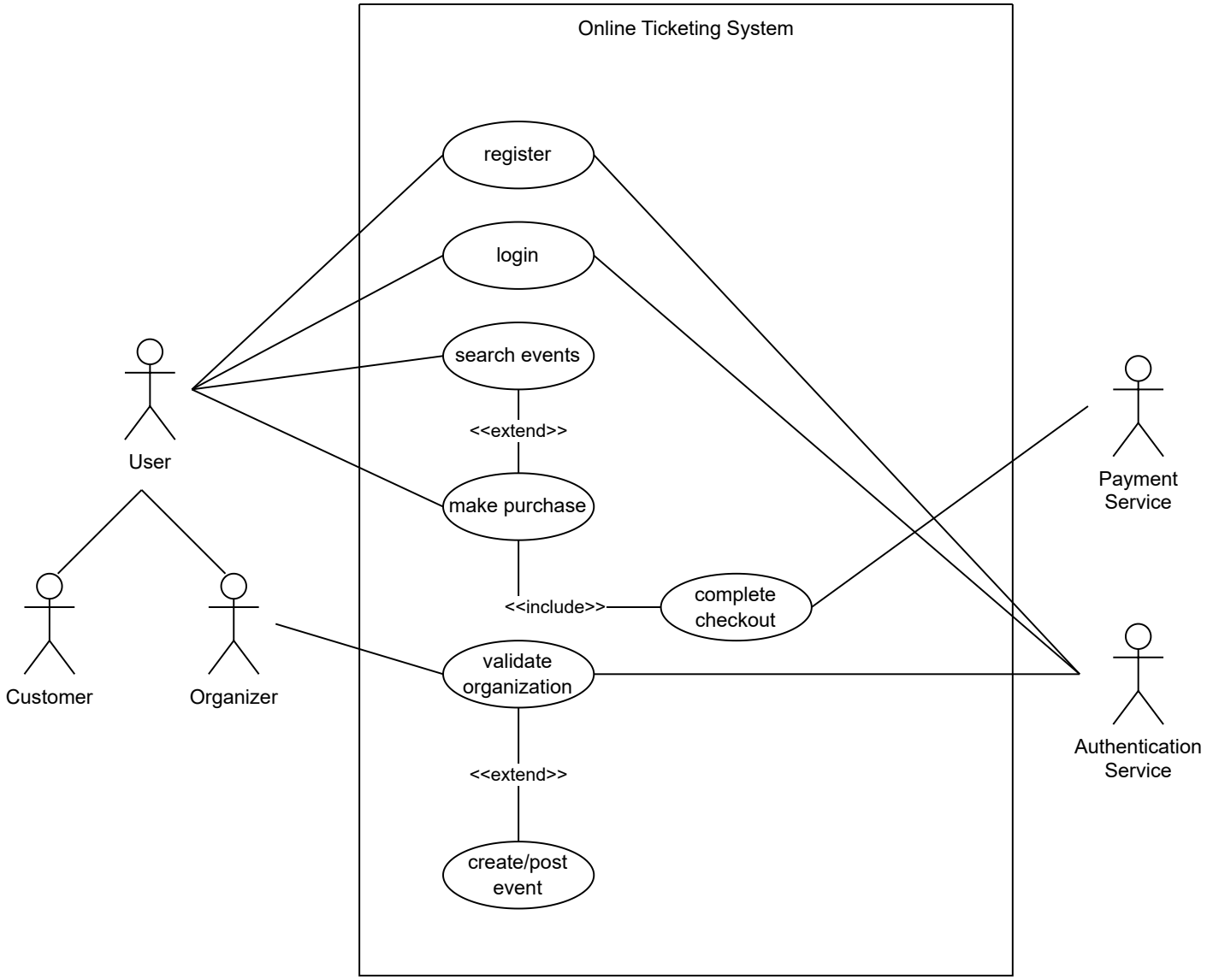

Diagram 2: Use-Case

Class Diagram

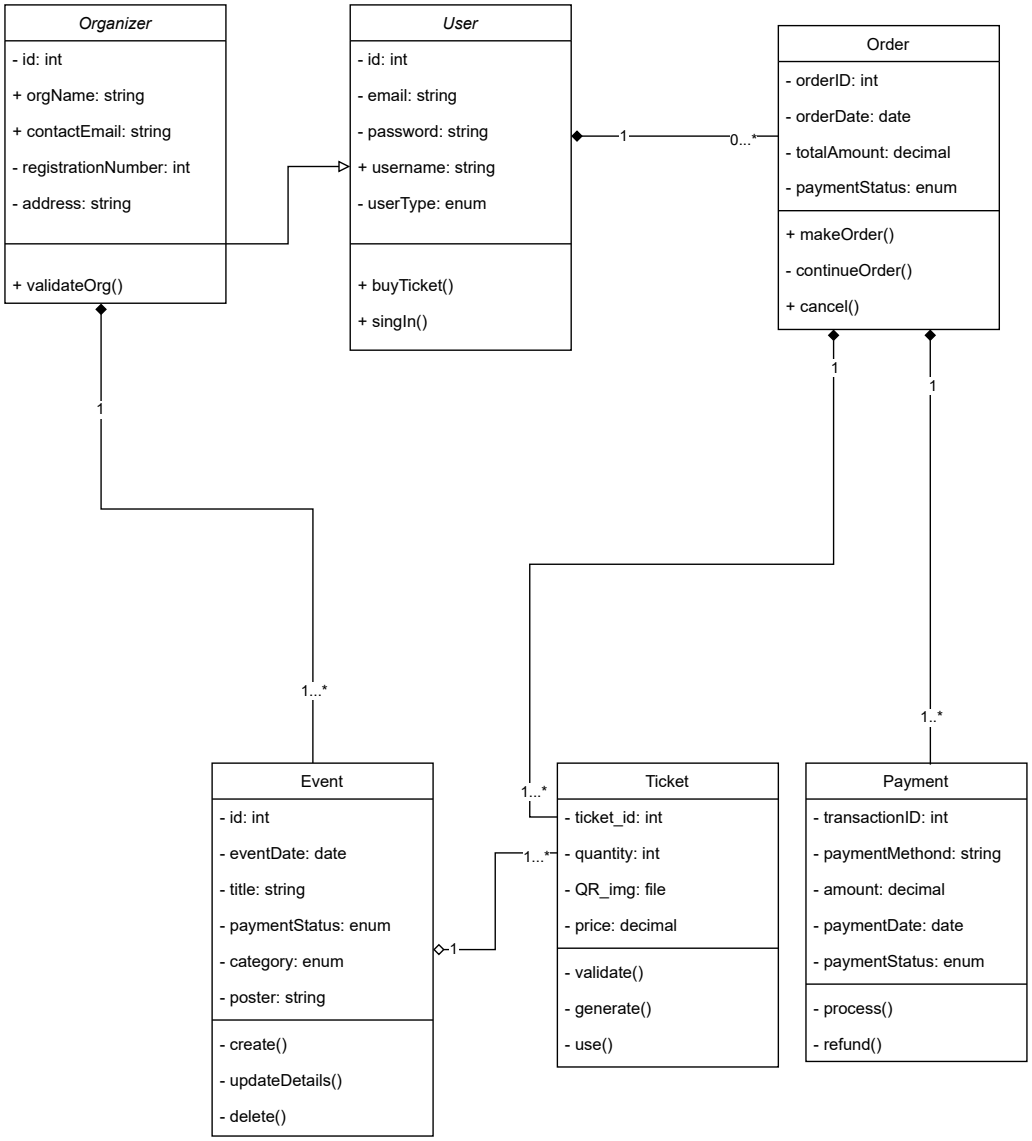

Diagram 3: Class

Sequence Diagram

This sequence diagram represents how Organizer/Host publishes an event in the system. Authentication service makes sure the legal info of the host is valid so that event publication is enabled afterwards.

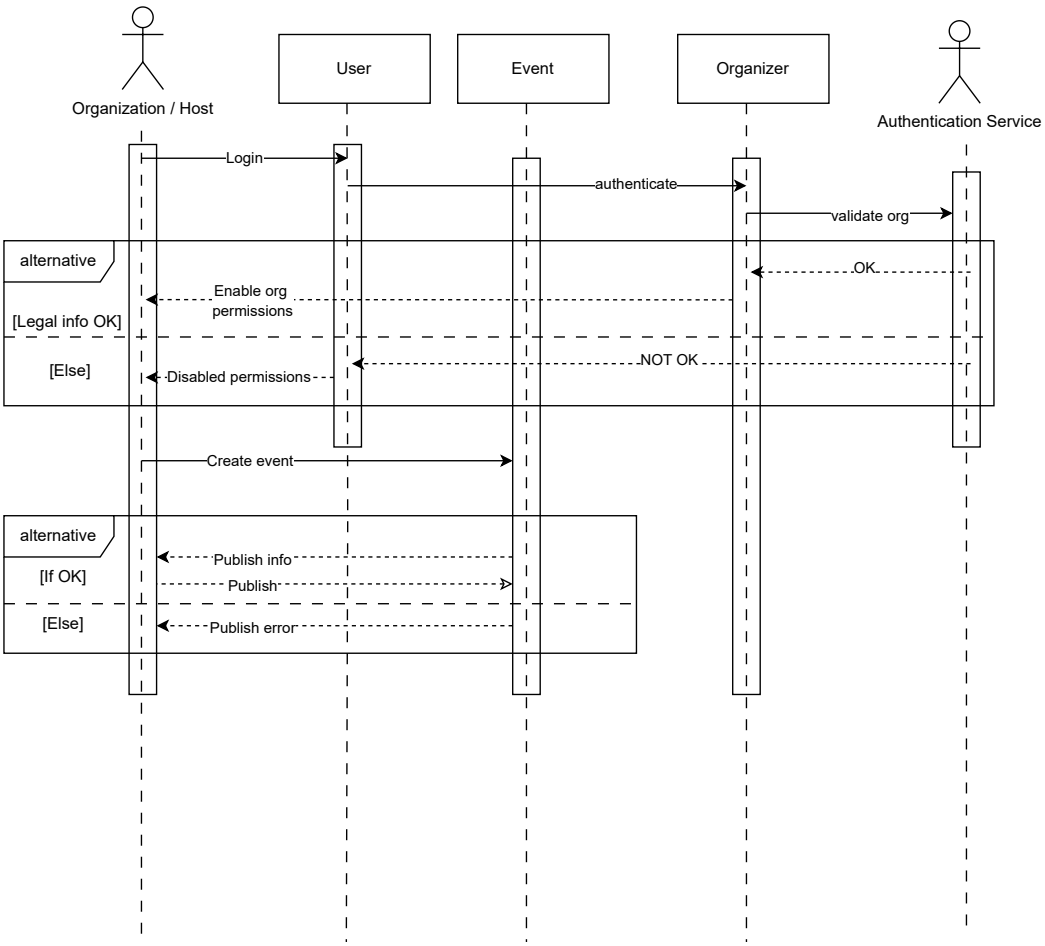

Diagram 4: Sequence